A4.3 AXI4 changes to memory attribute signaling

AXI4 makes the following changes to the AXI3 memory attribute signaling:

- the **AxCACHE[1]** bits are renamed as the *Modifiable* bits
- ordering requirements are defined for *Non-modifiable* transactions
- the meanings of *Read-allocate* and *Write-allocate* are updated.

A4.3.1 AxCACHE[1], Modifiable

In AXI4, the **AxCACHE[1]** bits are the *Modifiable* bits. When HIGH, Modifiable indicates that the characteristics of the transaction can be modified. When Modifiable is LOW, the transaction is *Non-modifiable*.

Note

The **AxCACHE[1]** bits are renamed from the *Cacheable* bits to the *Modifiable* bits to better describe the required functionality. The actual functionality is unchanged.

The following sections describe the properties of Non-modifiable and Modifiable transactions.

Non-modifiable transactions

A Non-modifiable transaction is indicated by setting **AxCACHE[1]** LOW.

A Non-modifiable transaction must not be split into multiple transactions or merged with other transactions.

In a Non-modifiable transaction, the parameters shown in Table A4-2 must not be changed.

Table A4-2 Parameters fixed as Non-modifiable

Parameter	Signals
Transfer address	**AxADDR**, and therefore **AxREGION**
Burst size	**AxSIZE**
Burst length	**AxLEN**
Burst type	**AxBURST**
Lock type	**AxLOCK**
Protection type	**AxPROT**

The **AxCACHE** attribute can only be modified to convert a transaction from being Bufferable to Non-bufferable. No other change to **AxCACHE** is permitted.

The transaction ID and the QoS values can be modified.

A Non-modifiable transaction with burst length greater than 16 can be split into multiple transactions. Each resulting transaction must meet the requirements given in this subsection, except that:

- the burst length is reduced
- the address of the generated bursts is adapted appropriately.

Note

There are circumstances where it is not possible to meet the requirements of Non-modifiable transactions. For example, when downsizing to a bus width narrower than that required by the transaction size, **AxSIZE**, the transaction must be modified.

A component that performs such an operation can optionally include an IMPLEMENTATION DEFINED mechanism to indicate that a modification has occurred. This can assist with software debug.

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

Modifiable transactions

A Modifiable transaction is indicated by asserting **AxCACHE[1]**.

A Modifiable transaction can be modified in the following ways:

- a transaction can be broken into multiple transactions
- multiple transactions can be merged into a single transaction
- a read transaction can fetch more data than required
- a write transaction can access a larger address range than required, using the **WSTRB** signals to ensure that only the appropriate locations are updated
- in each generated transaction, the following signals can be modified:
  - the transfer address, **AxADDR**
  - the burst size, **AxSIZE**
  - the burst length, **AxLEN**
  - the burst type, **AxBURST**.

The following must not be changed:

- the lock type, **AxLOCK**
- the protection type, **AxPROT**.

The memory attribute, **AxCACHE**, can be modified, but any modification must ensure that the visibility of transactions by other components is not reduced, either by preventing propagation of transactions to the required point, or by changing the need to look up a transaction in a cache. Any modification to the memory attributes must be consistent for all transactions to the same address range.

The transaction ID and QoS values can be modified.

No transaction modification is permitted that:

- Causes accesses to a different 4KByte address space than that of the original transaction.
- Causes a single access to a single-copy atomicity sized region to be performed as multiple accesses. See *Single-copy atomicity size* on page A7-90.

A4.3.2 Ordering requirements for Non-modifiable transactions

AXI4 requires that ordering is preserved for any set of transactions that meet all of the following conditions:

- the transactions are Non-modifiable
- the transactions use the same AXI ID
- the transactions target the same slave device.

The ordering must be preserved, irrespective of the address of the transaction, if the transactions are destined for the same slave.

Note

Ordering between the independent read and write channels can only be guaranteed if a transaction in one direction is issued only after any earlier transaction in the other direction has received a response. If a transaction in one direction is issued before receiving the response to any earlier transaction in the other direction, then no ordering exists between the transactions.

This ordering requirement makes no guarantee about the relative ordering of transactions destined for different slaves.

Because the address map boundary between different physical slave devices is IMPLEMENTATION DEFINED, if the boundary between slave devices is not known then the ordering of all Non-modifiable transactions with the same AXI ID on the same path must be preserved.

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

This ordering requirement applies between all Non-modifiable transactions, including between Non-bufferable and Bufferable transactions.

When an intermediate component in an AXI path issues transaction responses, that component is responsible for ensuring the correct ordering.

For more information on the ordering model see Chapter A6 *AXI4 Ordering Model*.

A4.3.3 Updated meaning of Read-allocate and Write-allocate

In AXI4, the meaning of the *Read-allocate* and *Write-allocate* bits is updated so that one bit indicates if an allocation occurs for the transaction and the other bit indicates if an allocation could have been made due to another transaction.

For read transactions, the write-allocate bit is redefined to indicate that:

- the location could have been previously allocated in the cache because of a write transaction (as the AXI3 definition)
- the location could have been previously allocated in the cache because of the actions of another master (additional AXI4 definition).

For write transactions, the read-allocate bit is redefined to indicate that:

- the location could have been previously allocated in the cache because of a read transaction (as the AXI3 definition)
- the location could have been previously allocated in the cache because of the actions of another master (additional AXI4 definition).

These changes mean:

- a transaction must be looked up in a cache if the value of **AxCACHE[3:2]** is not `0b00`
- a transaction does not need to be looked up in a cache if the value of **AxCACHE[3:2]** is `0b00`.

Note

The change to the definition of **AxCACHE** means that these signals can differ for a read and write transaction to the same location.

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

Table A4-3 shows the AXI4 bit allocations for the **AWCACHE** signals.

Table A4-3 AWCACHE bit allocations

Signal	AXI4 definition	Description
AWCACHE[3]	Allocate	When asserted, the transaction must be looked up in a cache because it could have been previously allocated. The transaction must also be looked up in a cache if **AWCACHE[2]** is asserted. When deasserted, if **AWCACHE[2]** is also deasserted, then the transaction does not need to be looked up in a cache and the transaction must propagate to the final destination. When asserted, it is recommended that this transaction is allocated in the cache for performance reasons.
AWCACHE[2]	Other Allocate	When asserted, the transaction must be looked up in a cache because it could have been previously allocated in the cache by another transaction, either a read transaction or a transaction from another master. The transaction must also be looked up in a cache if **AWCACHE[3]** is asserted. When deasserted, if **AWCACHE[3]** is also deasserted, then the transaction does not need to be looked up in a cache and the transaction must propagate to the final destination.
AWCACHE[1]	Modifiable	When asserted, the characteristics of the transaction can be modified and writes can be merged. When deasserted, the characteristics of the transaction must not be modified.
AWCACHE[0]	Bufferable	When deasserted, if both of **AWCACHE[3:2]** are deasserted, the write response must be given from the final destination. When asserted, if both of **AWCACHE[3:2]** are deasserted, the write response can be given from an intermediate point, but the write transaction is required to be made visible at the final destination in a timely manner. When deasserted, if either of **AWCACHE[3:2]** is asserted, the write response can be given from an intermediate point, but the write transaction is required to be made visible at the final destination in a timely manner. When asserted, if either of **AWCACHE[3:2]** is asserted, the write response can be given from an intermediate point. The write transaction is not required to be made visible at the final destination.

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

Table A4-4 shows the AXI4 bit allocations for the **ARCACHE** signals.

Table A4-4 ARCACHE bit allocations

Signal	AXI4 definition	Description
ARCACHE[3]	Other Allocate	When asserted, the transaction must be looked up in a cache because it could have been allocated in the cache by another transaction, either a write transaction or a transaction from another master. The transaction must also be looked up in a cache if **ARCACHE[2]** is asserted. When deasserted, if **ARCACHE[2]** is also deasserted, then the transaction does not need to be looked up in a cache.
ARCACHE[2]	Allocate	When asserted, the transaction must be looked up in a cache because it could have been allocated. The transaction must also be looked up in a cache if **ARCACHE[3]** is asserted. When deasserted, if **ARCACHE[3]** is also deasserted, then the transaction does not need to be looked up in a cache. When asserted, it is recommended that this transaction is allocated in the cache for performance reasons.
ARCACHE[1]	Modifiable	When asserted, the characteristics of the transaction can be modified and a larger quantity of read data can be fetched than is required. When deasserted the characteristics of the transaction must not be modified.
ARCACHE[0]	Bufferable	This bit has no effect when **ARCACHE[3:1]** are deasserted. When **ARCACHE[3:2]** are deasserted and **ARCACHE[1]** is asserted: • if this bit is deasserted, the read data must be obtained from the final destination • if this bit is asserted, the read data can be obtained from the final destination or from a write that is progressing to the final destination. When either **ARCACHE[3]** is asserted, or **ARCACHE[2]** is asserted, this bit can be used to distinguish between Write-through and Write-back memory types.

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential